Learning Parity with Physical Noise: Imperfections, Reductions and FPGA Prototype

Davide Bellizia[1] Clément Hoffmann[1] Dina Kamel[1] Hanlin Liu[2]
Pierrick Méaux[1] François-Xavier Standaert[1] Yu Yu[2]

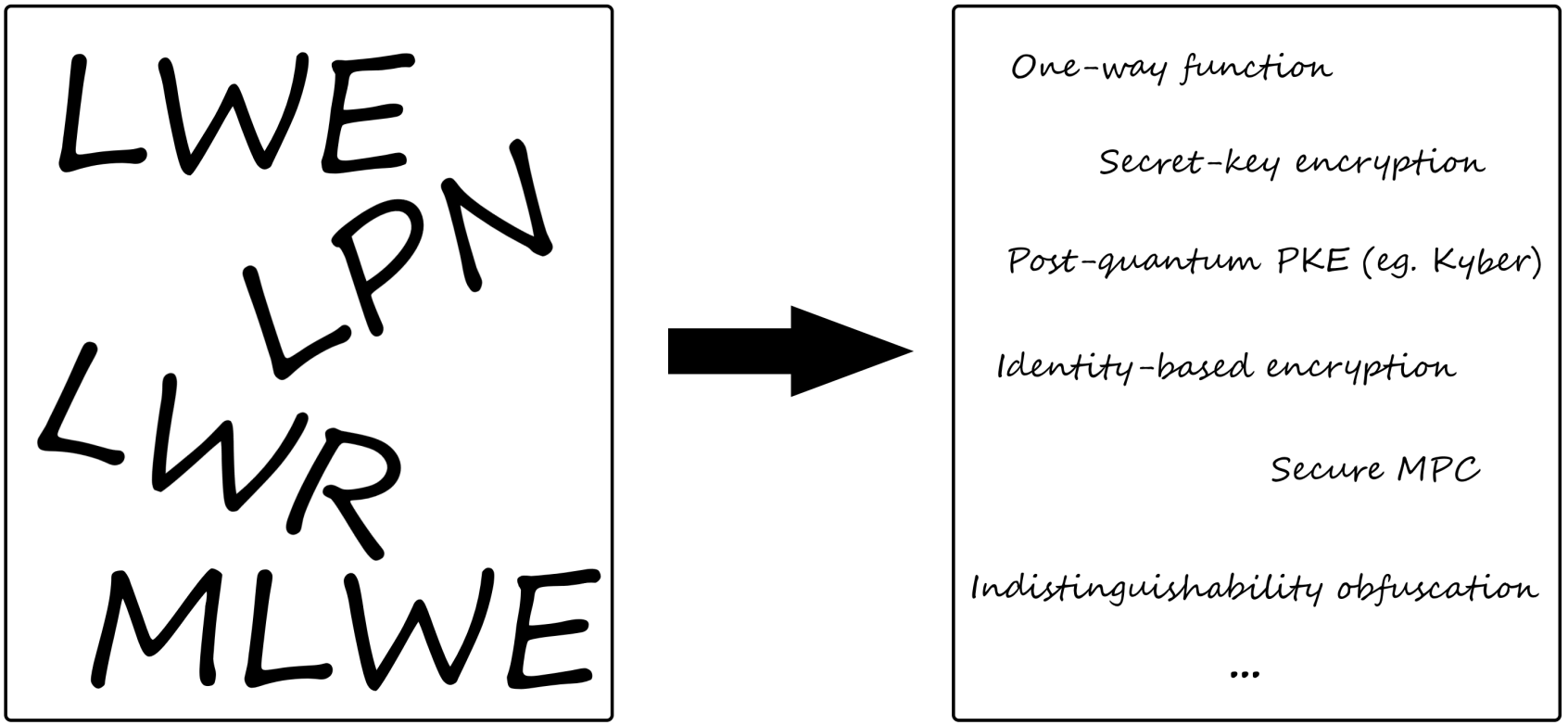
[1] UCLouvain, ICTEAM, Crypto Group, Louvain-la-Neuve, Belgium
[2] Shanghai Jiao Tong University, China

15th September 2021



Learning problems and LPN

Learning problems have proven to be interesting computationally hard problems.



The LPN problem



Among learning problems, **Learning with Parity Noise (LPN)** is the most straightforward one:

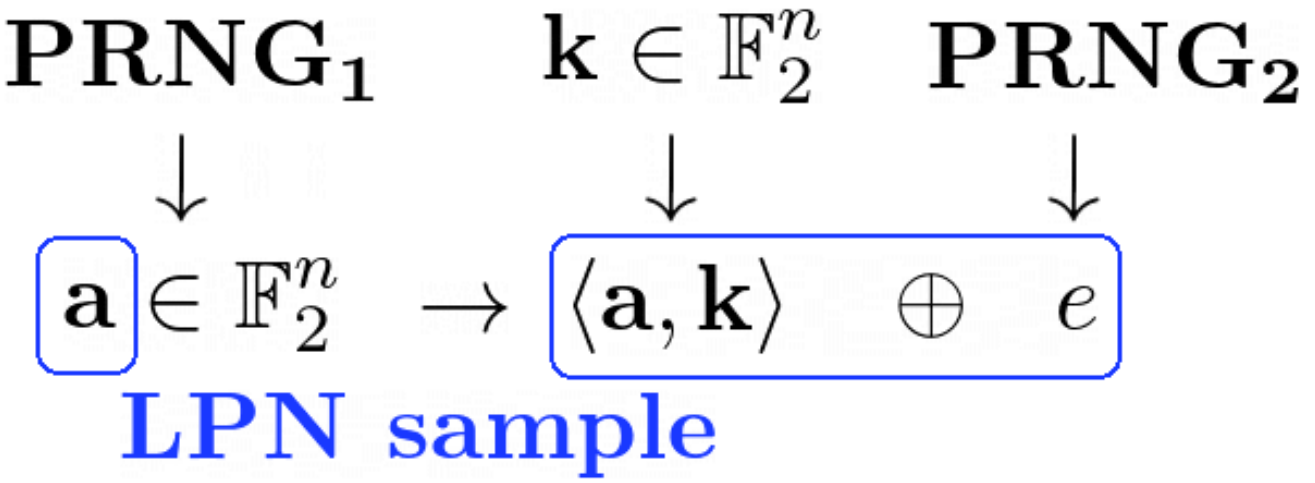
Learning with Parity Noise

Let $\boldsymbol{k}$ be a n-bit secret, $\epsilon \in [0, \frac{1}{2}]$ be a noise parameter.

Given $\begin{cases} \text{public } n\text{-bit vectors } (\boldsymbol{a_i})_i \\ \text{noisy } \mathbb{F}_2 \text{ inner product } \langle \boldsymbol{a_i}, \boldsymbol{k} \rangle \oplus e_i \text{ where } e_i \leftarrow \mathrm{Ber}_\varepsilon \end{cases}$,

it is computationally hard to retrieve $\boldsymbol{k}$.

LPN implementation



Unfortunately, the **error generation process** is expensive and appears to be a weak point against side-channel attacks in secure LPN implementations.

Learning with Physical Parity Noise (LPPN)



The **LPPN** problem is an approximation of the LPN problem. It uses **inexact computing** rather than a PRNG to generate error.
It therefore relies on a **physical assumption** and not a mathematical one.

We compute a **Physical Inner Product** (ϵ-PIP) using **voltage overscaling** and **clock manipulation**. Other methods exist to fault computations.

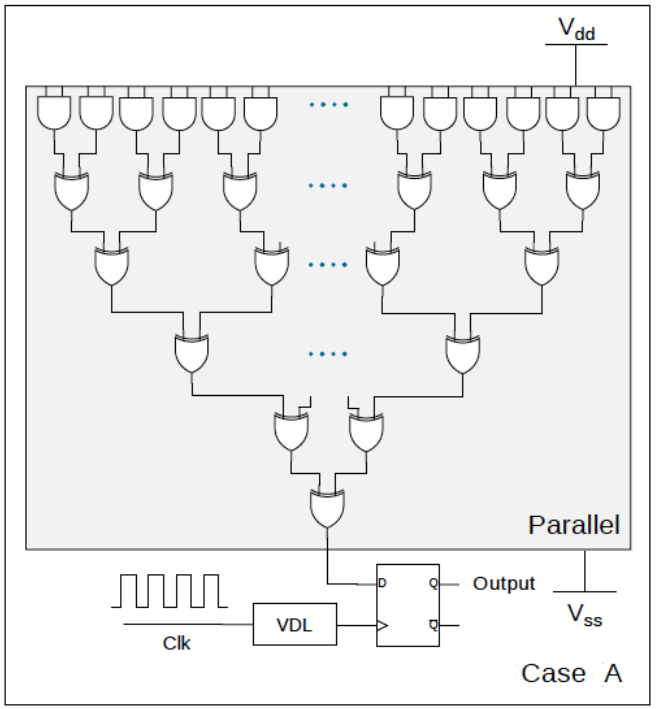
Outline



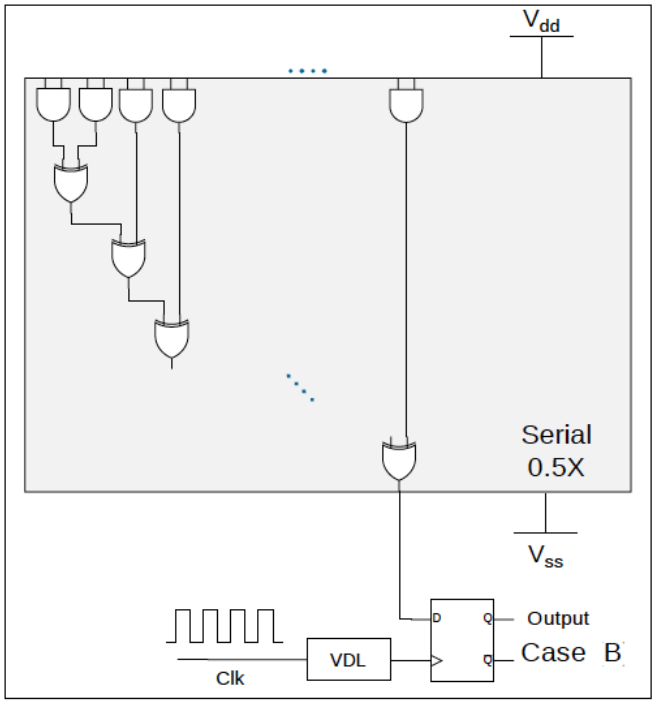
- **Study of limitations induced by inexact computing.**
- Security reductions asserting the security of our problem despite the limitations.
- FPGA prototype implementing a physical inner product.
- Other results.

Physical Inner Product architectures

In order to compute a **32-bit** Physical Inner Product (ϵ-PIP), we can use either a **parallel** (case A) or **serial** (case B) architecture.



A



B

Figure: ϵ-PIP architectures.

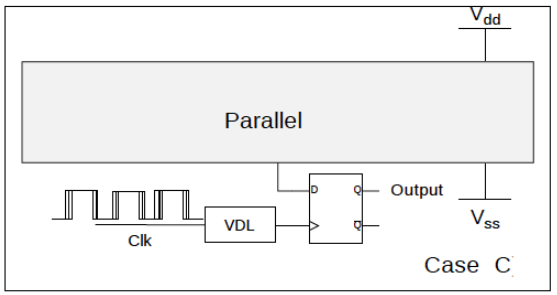
Output dependencies (1/3)



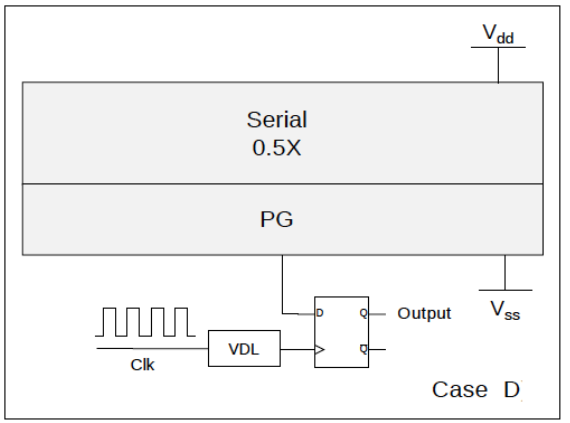
The probability of error generated by an ϵ-PIP depends on the correct output data.

For $i \in \{0,1\}, \epsilon_i = \Pr[err \mid \langle \boldsymbol{x}, \boldsymbol{k} \rangle = i]$.
ϵ_0 [resp. ϵ_1] is the probability of error of an ϵ-PIP where the correct output $\langle \boldsymbol{x}, \boldsymbol{k} \rangle = 0$ [resp. $\langle \boldsymbol{x}, \boldsymbol{k} \rangle = 1$], and $\Delta = \left| \frac{\epsilon_0 - \epsilon_1}{2} \right|$. We want our architecture to mitigate this Δ as much as possible.

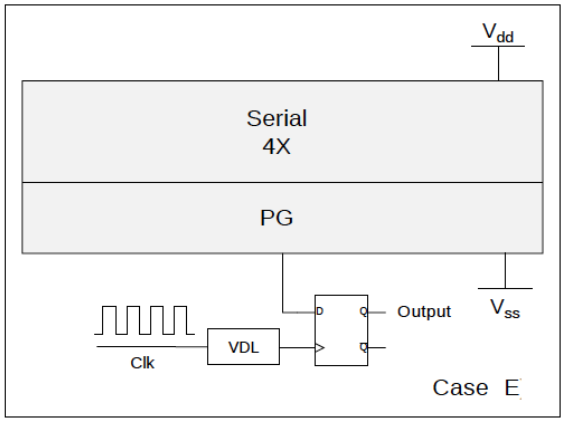
Output dependencies (2/3)



C



D



E

Figure: ϵ-PIP architectures.

Case C uses a parallel architecture with a jittery clock.
Case D uses a serial architecture using power gating.
Case E is the same as D except it uses bigger gates.

Output dependencies (3/3)

Here are the simulated Δ for the different cases:

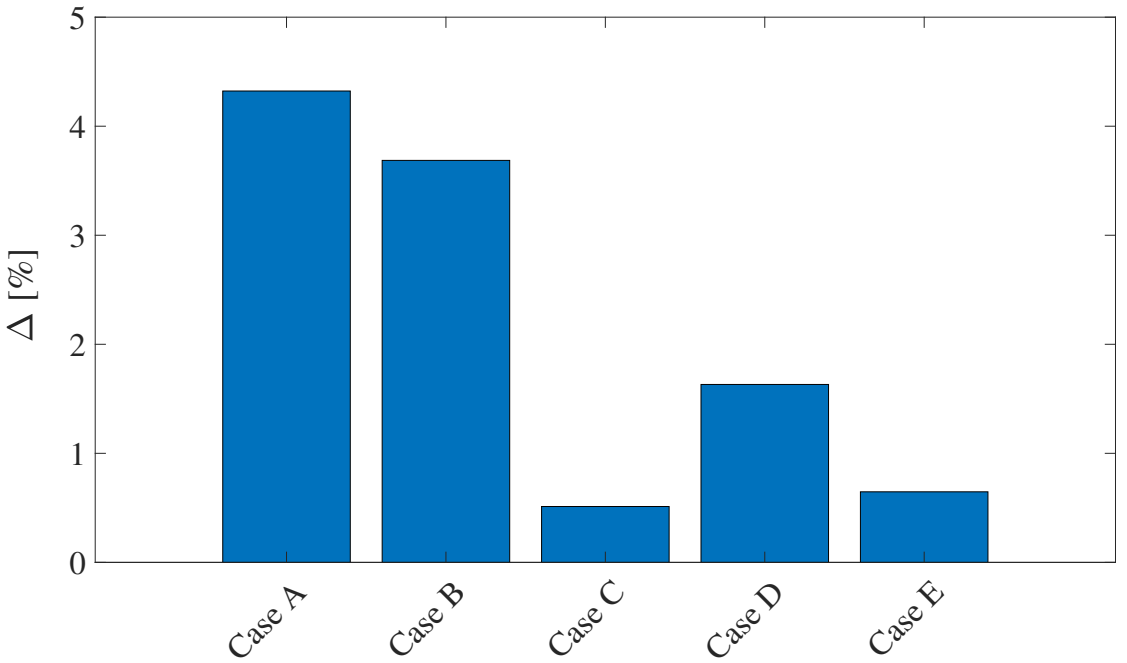


Figure: Architecture comparison

Can be reduced but not annihilated $\rightarrow$ motivates the next reduction...

Outline



- Study of limitations induced by inexact computing.
- **Security reductions asserting the security of our problem despite the limitations.**
- FPGA prototype implementing a physical inner product.
- Other results.

LPN-OD



Learning with Parity Noise with Output Dependencies (LPN-OD) is a theoretical modelisation of our physical problem.

The LPN-OD distribution

Let $\boldsymbol{k}$ be a n-bits secret, $\epsilon_0, \epsilon_1 \in [0, \frac{1}{2}]$ noise parameters.
We denote $\mathcal{D}_n^{\epsilon_0,\epsilon_1}$ the LPN-OD distribution defined as:

$$\left\{ (\boldsymbol{x}, \langle \boldsymbol{x}, \boldsymbol{k} \rangle \oplus e) : \boldsymbol{x} \leftarrow \mathbb{F}_2^n; e \leftarrow \left\{ \begin{array}{l} \mathrm{Ber}_{\epsilon_0} \text{ if } \langle \boldsymbol{x}, \boldsymbol{k} \rangle = 0 \\ \mathrm{Ber}_{\epsilon_1} \text{ if } \langle \boldsymbol{x}, \boldsymbol{k} \rangle = 1 \end{array} \right\} \right.$$

Security reduction



Note that $\mathrm{LPN}_{\epsilon}^{n}(\boldsymbol{k})$ is a particular case of $\mathrm{LPN\text{-}OD}_{\epsilon_0,\epsilon_1}^{n}(\boldsymbol{k})$, $(\epsilon_0 = \epsilon_1 = \epsilon)$.

We show that an adversary can transform LPN-OD samples into noisier ones, which gives a double security reduction (with some assumptions on the noise parameters $\epsilon, \epsilon_0, \epsilon_1$):

- An adversary using a LPN solver oracle can transform LPN-OD samples into LPN ones, and use its oracle.
  Therefore $\mathrm{LPN}_{\epsilon}^{n}(\boldsymbol{k}) \leq \mathrm{LPN\text{-}OD}_{\epsilon_0,\epsilon_1}^{n}(\boldsymbol{k})$.
- An adversary using a LPN-OD solver oracle can transform LPN samples into LPN-OD ones, and use its oracle.
  Therefore $\mathrm{LPN\text{-}OD}_{\epsilon_0,\epsilon_1}^{n}(\boldsymbol{k}) \leq \mathrm{LPN}_{\epsilon}^{n}(\boldsymbol{k})$.

Proof ideas



The idea is to build an algorithm that transform LPN-OD$^{n}_{\epsilon_0,\epsilon_1}(\boldsymbol{k})$ samples into noisier ones.

Algorithm 2: AltSampleLPN-OD$^{n}_{\epsilon_0,\epsilon_1,t_0,t_1,m}(\boldsymbol{k})$

Input: Error parameters $\epsilon_0, \epsilon_1 \in \left[0, \frac{1}{2}\right]$, Bernoulli parameters $t_0, t_1 \in \left[0, \frac{1}{2}\right]$ with $t_0 + t_1 \neq 1$

Output: A sample $(\boldsymbol{a}, b) \in \mathbb{F}_2^n \times \mathbb{F}_2$ following the distribution $D_{\boldsymbol{k}}^{\epsilon_0',\epsilon_1'}$

Proof ideas

The idea is to build an algorithm that transform LPN-OD$^n_{\epsilon_0,\epsilon_1}(\boldsymbol{k})$ samples into noisier ones.

This algorithm has three steps:

Check if the transformation is possible.

Algorithm 2: AltSampleLPN-OD$^n_{\epsilon_0,\epsilon_1,t_0,t_1,m}(\boldsymbol{k})$

Input: Error parameters $\epsilon_0, \epsilon_1 \in \left[0, \frac{1}{2}\right]$, Bernoulli parameters $t_0, t_1 \in \left[0, \frac{1}{2}\right]$ with $t_0 + t_1 \neq 1$

Output: A sample $(\boldsymbol{a}, b) \in \mathbb{F}_2^n \times \mathbb{F}_2$ following the distribution $D_{\boldsymbol{k}}^{\epsilon_0',\epsilon_1'}$

$\epsilon_0', \epsilon_1' = f(\varepsilon_0, \varepsilon_1, t_0, t_1)$;

if $\epsilon_0' \notin \left[0, \frac{1}{2}\right]$ *or* $\epsilon_1' \notin \left[0, \frac{1}{2}\right]$ **then**

  return $\perp$

end

Proof ideas



The idea is to build an algorithm that transform LPN-OD$^n_{\epsilon_0,\epsilon_1}(\boldsymbol{k})$ samples into noisier ones.

This algorithm has three steps:

- Check if the transformation is possible.
- **Simulate the result of the inner product.**

Algorithm 2: AltSampleLPN-OD$^n_{\epsilon_0,\epsilon_1,t_0,t_1,m}(\boldsymbol{k})$

Input: Error parameters $\epsilon_0, \epsilon_1 \in \left[0, \frac{1}{2}\right]$, Bernoulli parameters $t_0, t_1 \in \left[0, \frac{1}{2}\right]$ with $t_0 + t_1 \neq 1$

Output: A sample $(\boldsymbol{a}, b) \in \mathbb{F}_2^n \times \mathbb{F}_2$ following the distribution $D_{\boldsymbol{k}}^{\epsilon_0',\epsilon_1'}$

$\epsilon_0', \epsilon_1' = f(\varepsilon_0, \varepsilon_1, t_0, t_1)$;

if $\epsilon_0' \notin \left[0, \frac{1}{2}\right]$ *or* $\epsilon_1' \notin \left[0, \frac{1}{2}\right]$ **then**

  return $\perp$

end

$b' \xleftarrow{\$} \mathsf{Ber}_{\frac{1}{2}+\frac{\epsilon_0'-\epsilon_1'}{2}}$;

Proof ideas

The idea is to build an algorithm that transform LPN-OD$^n_{\epsilon_0,\epsilon_1}(\boldsymbol{k})$ samples into noisier ones.

This algorithm has three steps:

- Check if the transformation is possible.
- Simulate the result of the inner product.
- **Use rejection sampling to build the rest of the sample.**

Algorithm 2: AltSampleLPN-OD$^n_{\epsilon_0,\epsilon_1,t_0,t_1,m}(\boldsymbol{k})$

Input: Error parameters $\epsilon_0, \epsilon_1 \in \left[0, \frac{1}{2}\right]$, Bernoulli parameters $t_0, t_1 \in \left[0, \frac{1}{2}\right]$ with $t_0 + t_1 \neq 1$

Output: A sample $(\boldsymbol{a}, b) \in \mathbb{F}_2^n \times \mathbb{F}_2$ following the distribution $D_{\boldsymbol{k}}^{\epsilon'_0,\epsilon'_1}$

$\epsilon'_0, \epsilon'_1 = f(\varepsilon_0, \varepsilon_1, t_0, t_1)$;

if $\epsilon'_0 \notin \left[0, \frac{1}{2}\right]$ *or* $\epsilon'_1 \notin \left[0, \frac{1}{2}\right]$ **then**

  return $\perp$

end

$b' \xleftarrow{\$} \mathsf{Ber}_{\frac{1}{2}+\frac{\epsilon'_0-\epsilon'_1}{2}}$;

do

  $(\boldsymbol{a}, b'') \xleftarrow{\$} \text{SampleLPN-OD}^n_{\epsilon_0,\epsilon_1}(\boldsymbol{k})$;

  $e' \xleftarrow{\$} \mathsf{Ber}_{t_{b'}}$;

  $(\boldsymbol{a}, b) \leftarrow (\boldsymbol{a}, b'' + e')$;

while $b \neq b'$ *and* $cnt \leq m$;

return $(\boldsymbol{a}, b)$

Concrete security estimation



Using our reduction, we can quantify the security of LPN-OD using the one of LPN:

$\text{LPN-OD}^n_{\epsilon' \pm \Delta, \epsilon' \mp \Delta}(\boldsymbol{k})$ is at least as hard as $\text{LPN}^n_{\epsilon}(\boldsymbol{k})$, where

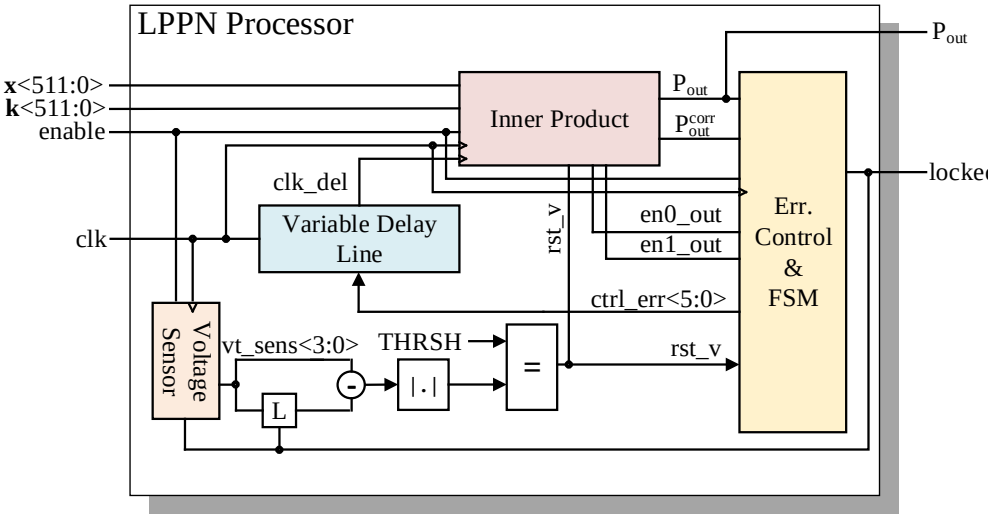
$$\epsilon = \frac{\epsilon' - \Delta}{1 - 2\Delta}$$

Outline



- Study of limitations induced by inexact computing.
- Security reductions asserting the security of our problem despite the limitations.
- **FPGA prototype implementing a physical inner product.**
- Other results.

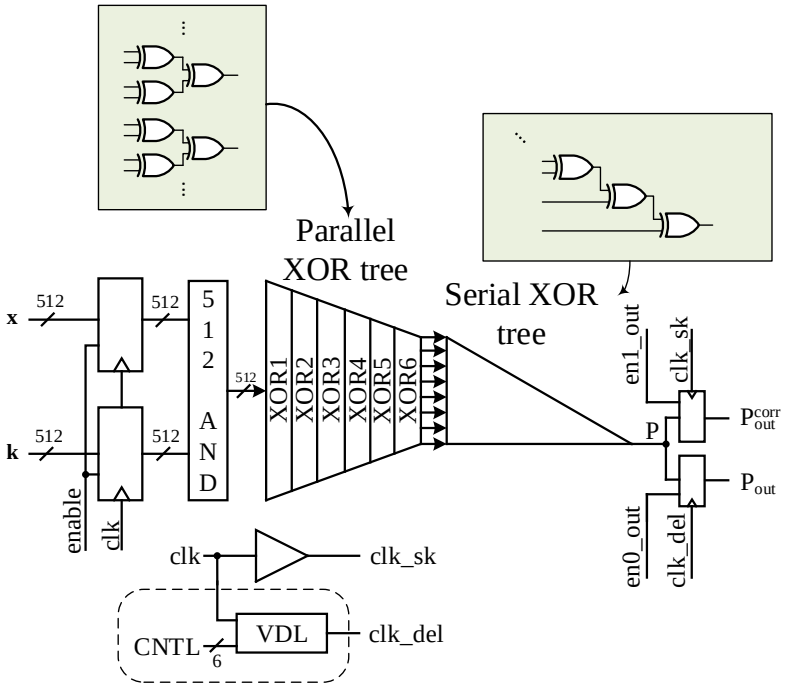
FPGA Prototype

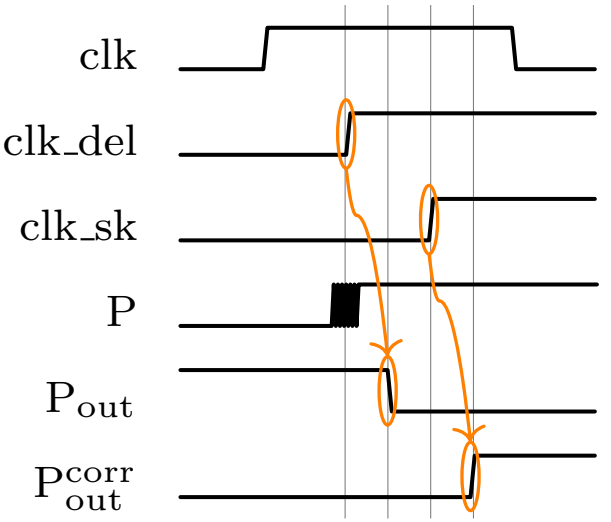


Module	LUTs	Regs
LPPN	1290 (100%)	1086 (100%)
Inner Product	1024 (79.4%)	1027 (94.6%)
VDL	92 (7.1%)	-
Error Control	50 (3.9%)	39 (3.6%)
Voltage Sensor	104 (8.1%)	8 (0.7%)
Others	20 (1.5%)	12 (1.1%)

- Full digital design of the LPPN processor;
- Targeting a Xilinx Spartan-6 LX75 mounted on a Sakura-G board;
- No special purpose macros required (e.g. PLLs, DCMs, etc.):
  - Only fabric elements from the FPGA have been used.

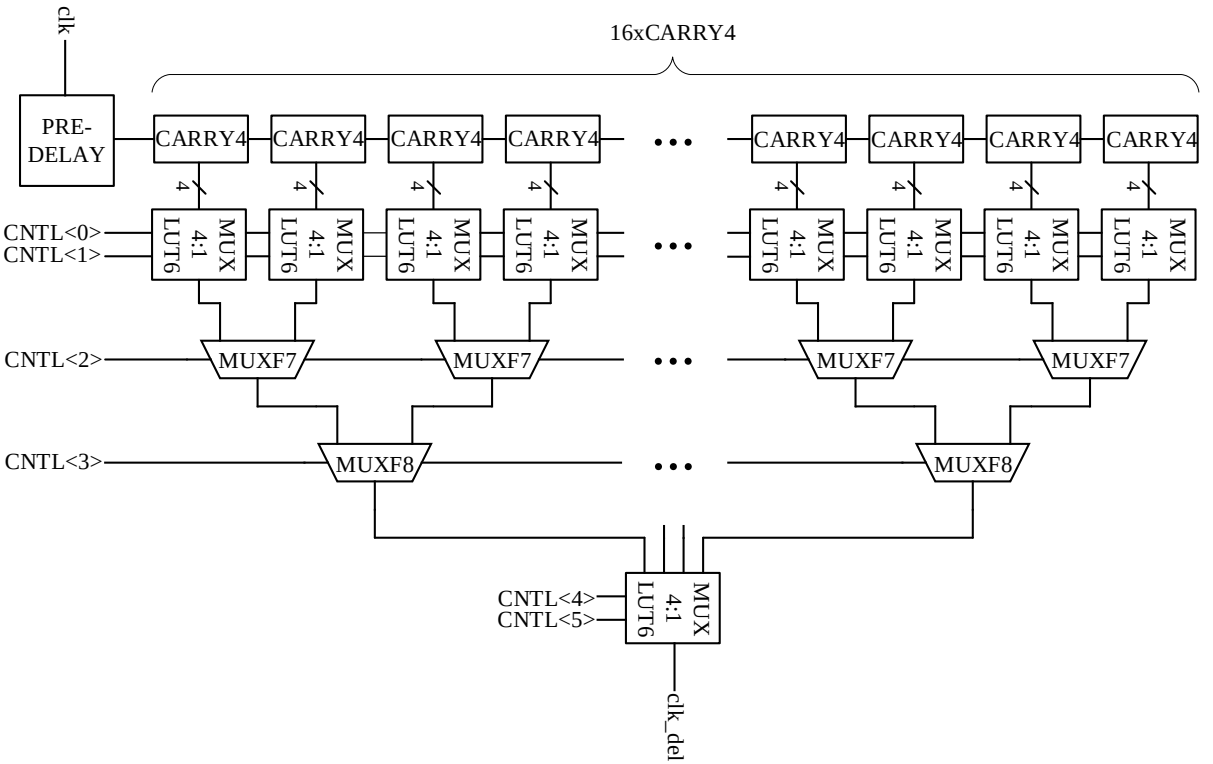
FPGA Prototype





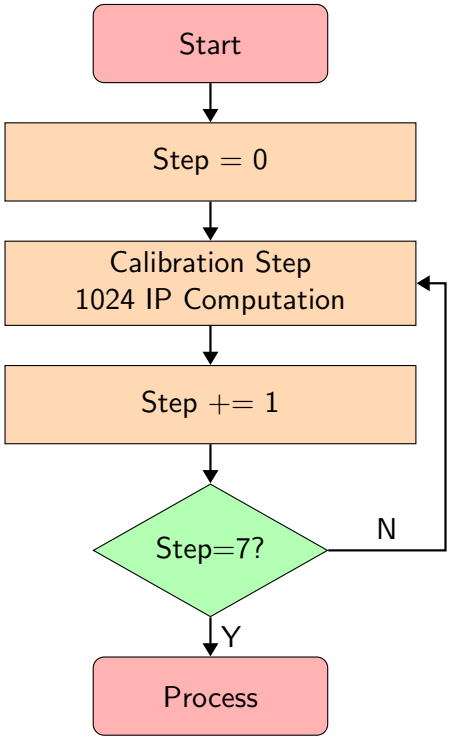
- **Inner Product (IP):** composition of a *parallel XOR tree* and a *serial XOR tree*;
- **Inexact Computing:** capturing metastable state and glitching events on the output of the IP block to generate errors.

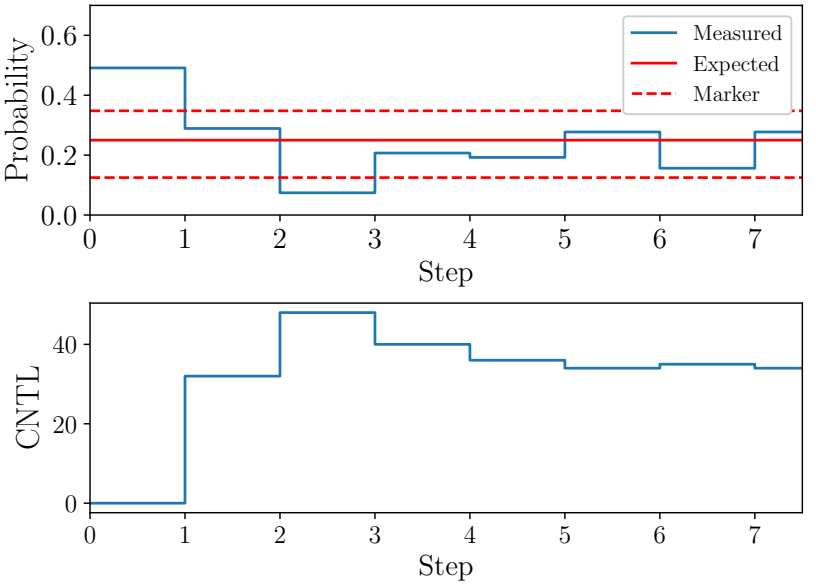
FPGA Prototype



- Compact and full-digital delay line composed by 16 CARRY4 units;
  - CARRY4 are fast chains of XOR in SLICE M/L used to implement arithmetic functions;
- 64 taps offering ∼30ps per tap;
- Very compact 64:1 mux implemented on LUTs and in-SLICE F7/F8 muxes.

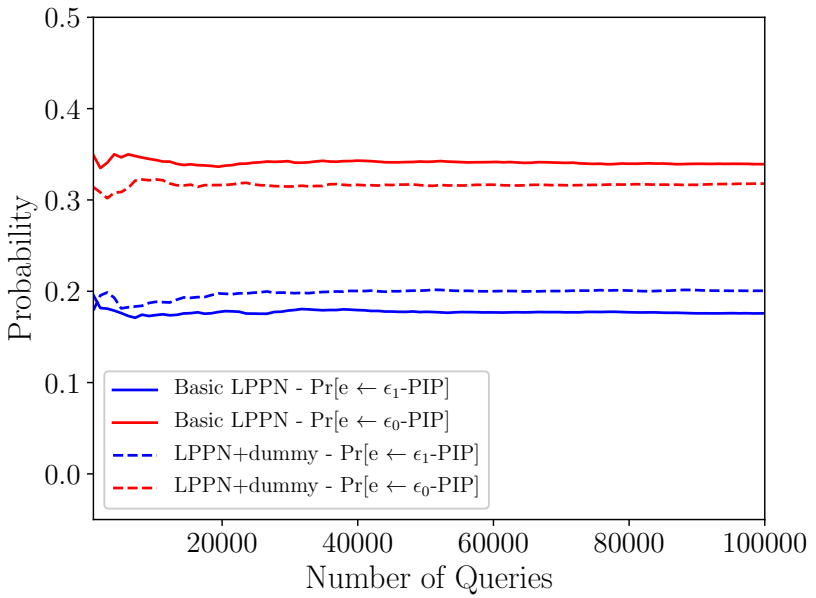
FPGA Prototype





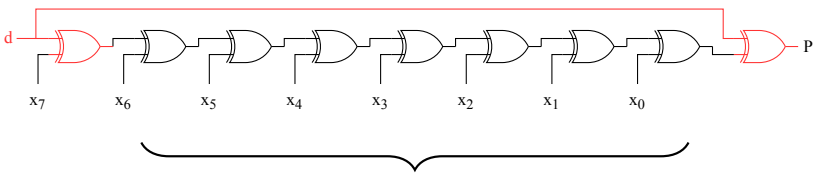
- The **Error Control Module** is implemented as a *Successive Approximation Register* (SAR) to calibrate the LPPN at the start-up:
  - 7 batches of 1024 IPs are computed;
  - at the end of each batch, the $\Pr[\epsilon = 1]$ is computed and the CNTL word is updated from MSB to LSB.
- After the calibration the error probability is $\Pr[\epsilon = 1] \approx 0.25$.

Output Dependencies



	Basic LPPN	LPPN+dummy
$\Pr[e \leftarrow \epsilon_1 - PIP]$	0.0	0.0
$\Pr[e \leftarrow \epsilon_0 - PIP]$	0.0	0.0
Δ	8.2%	5.8%

- The Δ=8.2% indicates that the prototype suffers from non negligible output dependencies.
- To mitigate such dependencies we complemented the basic design with an additional **dummy circuit** to provide data-independent glitches:



Serial XOR tree

- Adopting this solution, the normalized difference is reduced to Δ=5.8%.

Putting things together



$\text{LPN-OD}^{n}_{\epsilon'\pm\Delta,\epsilon'\mp\Delta}(\boldsymbol{k})$ is at least as hard as $\text{LPN}^{n}_{\epsilon}(\boldsymbol{k})$, where $\epsilon = \frac{\epsilon'-\Delta}{1-2\Delta}$.

Concretely, for the $\Delta \approx 8.2\%$ around an error probability of 0.25 reached in the FPGA prototype (512 bits inner product), we have a $\text{LPN-OD}^{512}_{22.95,27.05}(\boldsymbol{k})$ which security reduces to a $\text{LPN}^{512}_{0.20}(\boldsymbol{k})$.

Using the best known-attack against LPN, this gives us at least **80 bits of security** for a scheme which would rely on our implementation.

Outline



- Study of limitations induced by inexact computing.
- Security reductions asserting the security of our problem despite the limitations.
- FPGA prototype implementing a physical inner product.
- **Other results.**

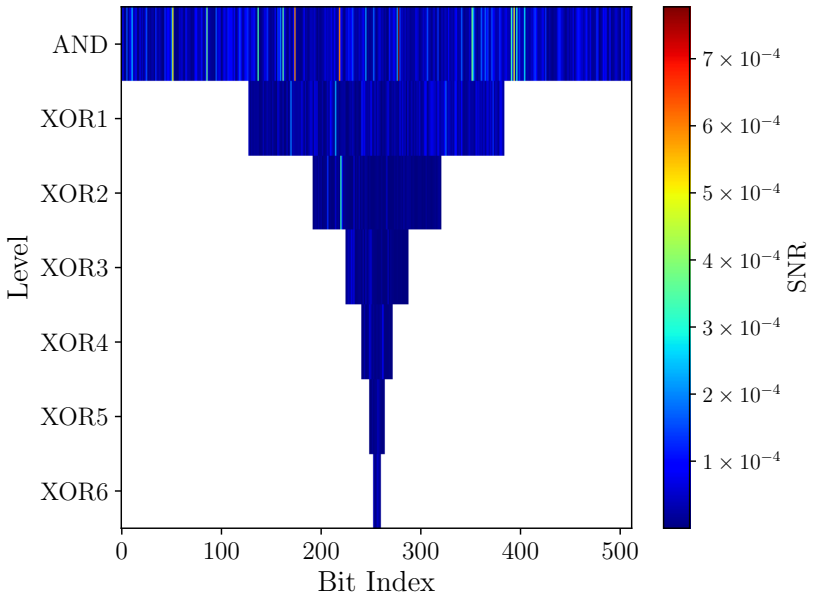
Masked LPPN

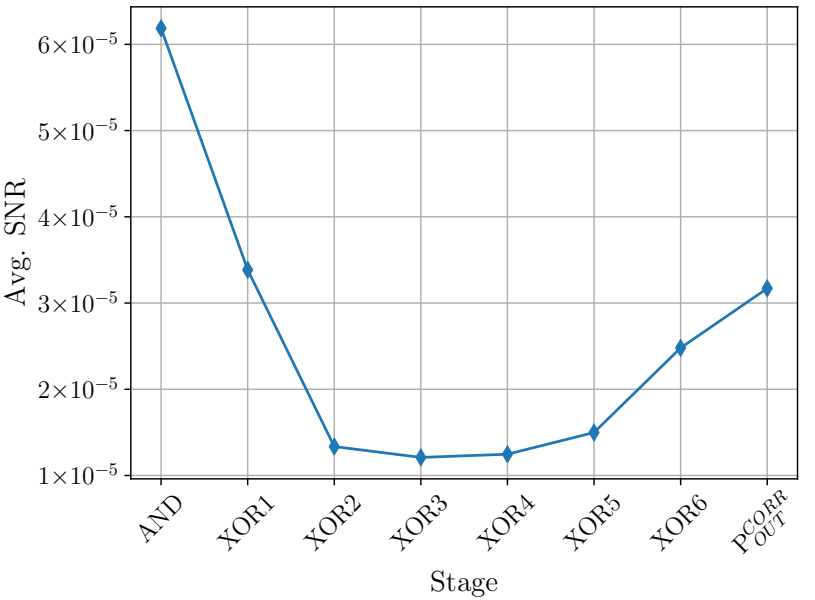
$$\cancel{y = \epsilon\text{-PIP}(\boldsymbol{x}, \boldsymbol{k})} \longrightarrow y = \epsilon\text{-PIP}(\boldsymbol{x}, \boldsymbol{k}_1) + \langle \boldsymbol{x}, \boldsymbol{k}_2 \rangle + \cdots + \langle \boldsymbol{x}, \boldsymbol{k_d} \rangle$$

Masking helps against **Differential Power Analysis** (DPA) and **structured error**, but also mitigates Δ.

Indeed, an adversary does not have access to $z = \epsilon\text{-PIP}(\boldsymbol{x}, \boldsymbol{k}_1)$ but only its leakage $\boldsymbol{l}$ ($\Pr[z = i \mid \boldsymbol{l}] = \frac{1}{2} + \delta$). The only imbalance that an adversary can oberve will therefore be $\begin{cases} \epsilon'_0 = \epsilon_0 \cdot \left(\frac{1}{2} + \delta\right) + \epsilon_1 \cdot \left(\frac{1}{2} - \delta\right) \\ \epsilon'_1 = \epsilon_1 \cdot \left(\frac{1}{2} + \delta\right) + \epsilon_0 \cdot \left(\frac{1}{2} - \delta\right) \end{cases}$.

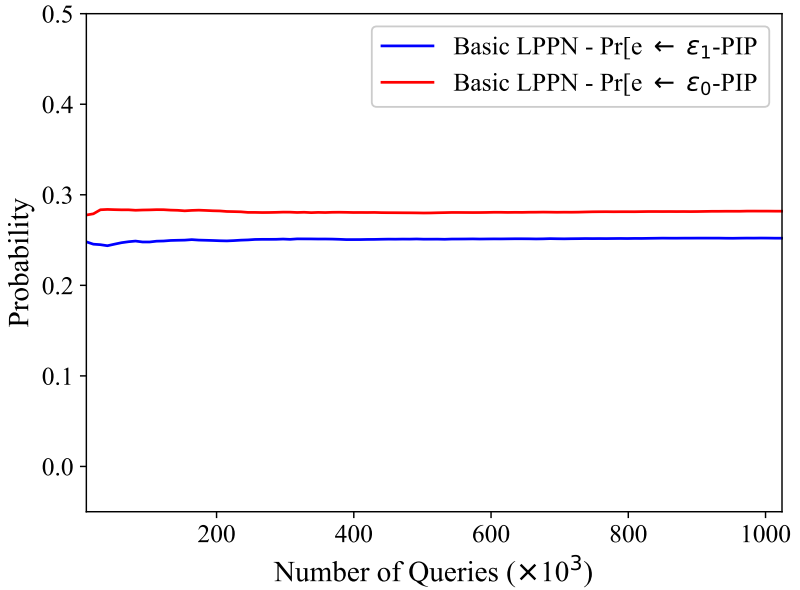
Side-Channel Evaluation





- We collected 4×10^6 power consumption traces to estimate Mangard's SNR.
- Highest SNR values have been found on the first stage (AND layer), where the dependency on the secret is still quite high.
- Yet, highest peaks of SNR are in the order of 10^{-5}:
  - Boolean masking is an interesting candidate for countermeasures that can efficiently leverage on key homomorphic features of the LPPN.

Dependencies on Artix-7



Bits fixed to 0	MSB	LSB
0	0.2735	0.2735
64	0.2547	0.2659
96	0.2565	0.2517
128	0.2577	0.0
256	0.2485	0.0
384	0.2803	0.0

- We ported the basic LPPN design to a more recent (and actual) platform:
  - Xilinx Artix-7 100T (28nm technology), mounted on a NewAE CW305 board.
- For this implementation we have Δ=1.5%, that is **strongly reduced** compared to the Spartan-6 design.
- Such reduction confirms the technological dependency of this aspect.
- Input dependency analysis highlights that the Artix-7 implementation can tolerate up to 96 bits by a filtering attack.

Conclusion



Inexact computing applied to LPN:

Pros	**Cons**
Good performances	Relies on physical assumption
Good features for SCA protection	Induces data dependant error

Security reduction handles the data dependant errors, which are themselves being reduced by smart hardware implementation.

Conclusion



Inexact computing applied to LPN:

Pros	**Cons**
Good performances	Relies on physical assumption
Good features for SCA protection	Induces data dependant error

Security reduction handles the data dependant errors, which are themselves being reduced by smart hardware implementation.

Next step: Investigate other learning problems (e.g., LWE) in order to build more advanced primitives.